

Statistical and Machine Learning

Neural Networks for Sequences

Katarzyna Reluga

Neural Networks for Sequences

- ▶ Introduction
- ▶ Recurrent Neural Networks (RNNs)
- ▶ 1d CNNs
- ▶ Attention
- ▶ Transformers
- ▶ Language Models

Neural Networks for Sequences

- ▶ **Introduction**
- ▶ Recurrent Neural Networks (RNNs)
- ▶ 1d CNNs
- ▶ Attention
- ▶ Transformers
- ▶ Language Models

Introduction

- ▶ We look at neural networks designed for sequences, where the input, the output, or both can be sequences.
- ▶ These models are used in a wide range of applications: machine translation, speech recognition, text classification, image captioning, and more.
- ▶ The presentation here follows closely [PML: Chapter 15](#) which is based on Zhang et al. (2020), which is a good reference for further detail.

Neural Networks for Sequences

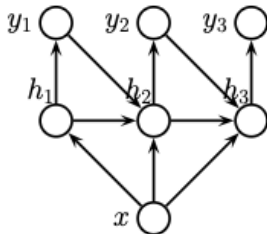
- ▶ Introduction
- ▶ **Recurrent Neural Networks (RNNs)**
- ▶ 1d CNNs
- ▶ Attention
- ▶ Transformers
- ▶ Language Models

Recurrent Neural Networks (RNNs)

- ▶ A **recurrent neural network (RNN)** maps an input space of sequences to an output space of sequences in a *stateful* way.
- ▶ The prediction of $\mathbf{y}_t$ depends not just on the input $\mathbf{x}_t$, but also on a hidden state $\mathbf{h}_t$, which is updated as the sequence is processed.
- ▶ This hidden state acts as a form of memory, and lets us build models for sequence generation, sequence classification, and sequence translation.

Vec2Seq: Sequence Generation

- ▶ Here we want to learn $f_{\theta} : \mathbb{R}^D \rightarrow \mathbb{R}^{N_{\infty} C}$, where D is the input size and the output is an arbitrary-length sequence of vectors of size C .
- ▶ Because they map a single vector to a sequence, we call these **vec2seq** models.
- ▶ The output sequence $\mathbf{y}_{1:T}$ is generated one token at a time: we sample $\tilde{y}_t$ from the hidden state $\mathbf{h}_t$, then feed it back in to get the new state $\mathbf{h}_{t+1}$ (see Figure below)
- ▶ This defines a conditional generative model $p(\mathbf{y}_{1:T}|\mathbf{x})$, which captures dependencies between the output tokens.



Vec2Seq: Model Definition

- ▶ The generative process can be written explicitly as

$$\begin{aligned} p(\mathbf{y}_{1:T}|\mathbf{x}) &= \sum_{\mathbf{h}_{1:T}} p(\mathbf{y}_{1:T}, \mathbf{h}_{1:T}|\mathbf{x}) \\ &= \sum_{\mathbf{h}_{1:T}} \prod_{t=1}^T p(\mathbf{y}_t|\mathbf{h}_t) p(\mathbf{h}_t|\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) \end{aligned}$$

where $\mathbf{h}_t$ is the hidden state, and let $p(\mathbf{h}_1|\mathbf{h}_0, \mathbf{y}_0, \mathbf{x}) = p(\mathbf{h}_1|\mathbf{x})$ be the initial state distribution (often deterministic).

- ▶ Let $c = 1, \dots, C$. For categorical outputs, e.g., words, we use

$$p(\mathbf{y}_t|\mathbf{h}_t) = \text{Cat}(\mathbf{y}_t | \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y))$$

where $\text{Cat}(\cdot)$ stands for the multinomial distribution, and $\mathbf{W}_{hy}$ are the hidden-to-output weights, $\mathbf{b}_y$ and $\text{softmax}(\mathbf{z}_c) = \frac{\exp(\mathbf{z}_c)}{\sum_{k=1}^C \exp(\mathbf{z}_k)}$.

- ▶ For real-valued outputs, we instead use a Gaussian:

$$p(\mathbf{y}_t|\mathbf{h}_t) = \mathcal{N}(\mathbf{y}_t | \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \sigma^2\mathbf{I})$$

Vec2Seq: Hidden State Update

- ▶ The hidden state is computed deterministically, given the past state, past output, and input:

$$p(\mathbf{h}_t | \mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}) = \mathbb{I}(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{y}_{t-1}, \mathbf{x}))$$

- ▶ The update function f usually takes the form

$$\mathbf{h}_t = \varphi(\mathbf{W}_{xh}[\mathbf{x}; \mathbf{y}_{t-1}] + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

with $\mathbf{W}_{hh}$ the hidden-to-hidden weights and $\mathbf{W}_{xh}$ the input-to-hidden weights.

- ▶ Since $\mathbf{y}_t$ depends on $\mathbf{h}_t$, which depends on $\mathbf{y}_{t-1}$, and so on, $\mathbf{y}_t$ implicitly depends on all past observations.

Vec2Seq: Generation Process

- ▶ In practice, the effective memory length is determined by the size of the latent state and the strength of the parameters.
- ▶ To generate from RNN we sample from $\tilde{\mathbf{y}}_t \sim p(\mathbf{y}_t | \mathbf{h}_t)$, then “feed it in” to the hidden state, to deterministically compute $\mathbf{h}_{t+1} = f(\mathbf{h}_t, \tilde{\mathbf{y}}_t, \mathbf{x})$, from which we sample, $\tilde{\mathbf{y}}_{t+1} \sim p(\mathbf{y}_{t+1} | \mathbf{h}_{t+1})$ and repeat.
- ▶ The transition $\mathbf{h}_{t+1} = f(\mathbf{h}_t, \tilde{\mathbf{y}}_t, \mathbf{x})$ is deterministic given the previous hidden state, input, and sampled output. Therefore, the only stochasticity in generation comes from the emission model $\tilde{\mathbf{y}}_t \sim p(\mathbf{y}_t | \mathbf{h}_t)$, and the sampled output $\tilde{\mathbf{y}}_t$ is fed back into the next hidden-state update.

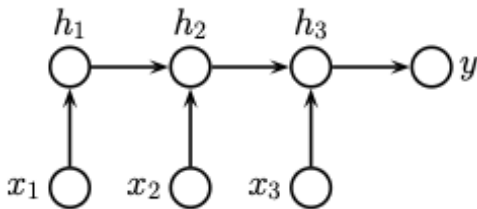
Applications: Language Modeling

- ▶ RNNs can generate sequences unconditionally, by setting $\mathbf{x} = \emptyset$, or conditionally on some input $\mathbf{x}$.
- ▶ Unconditional generation is called **language modeling**: learning a joint distribution $p(y_1, \dots, y_T)$ over sequences of tokens.
- ▶ Example: output of length 500 generated from an RNN trained on *The Time Machine* by H.G. Wells – plausible-looking, but not always meaningful, text.

the githa some thong the time traveller held in his hand was a glitteringmetallic framework scarcely larger than a small clock and verydelicately made there was ivory in it and the latter than s bettyre tat howhong s ie time thave ler simk you a dimensions le ghat dionthat shall travel indifferently in any direction of space and timeas the driver determinesfilby contented himself with laughterbut i have experimental verification said the time travellerit would be remarkably convenient for the histo

Seq2Vec: Sequence Classification and Model

- ▶ In sequence classification, or **seq2vec**, we want a single fixed-length output $\mathbf{y}$ given a variable-length input sequence.
- ▶ We want to learn $f_{\theta} : \mathbb{R}^{TD} \rightarrow \mathbb{R}^C$.
- ▶ For simplicity, we focus on the case where the output is a class label, $y \in \{1, \dots, C\}$.
- ▶ The simplest approach uses the final hidden state of the RNN as input to the classifier: $p(y|\mathbf{x}_{1:T}) = \text{Cat}(y | \text{softmax}(\mathbf{W}\mathbf{h}_T))$

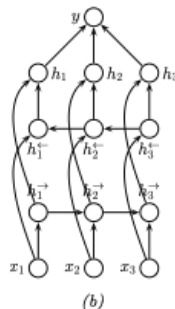
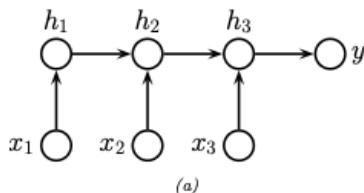


Bidirectional RNN

- ▶ We often get better results by letting the hidden states depend on both past and future context, not just the past.
- ▶ To do this, we build **bidirectional RNN** composed of two RNNs: one computes hidden states moving forwards through the sequence, and the other – moving backwards.
- ▶ The forward and backward passes are defined as
$$\mathbf{h}_t^{\rightarrow} = \varphi(\mathbf{W}_{xh}^{\rightarrow} \mathbf{x}_t + \mathbf{W}_{hh}^{\rightarrow} \mathbf{h}_{t-1}^{\rightarrow} + \mathbf{b}_h^{\rightarrow}) \text{ and}$$
$$\mathbf{h}_t^{\leftarrow} = \varphi(\mathbf{W}_{xh}^{\leftarrow} \mathbf{x}_t + \mathbf{W}_{hh}^{\leftarrow} \mathbf{h}_{t+1}^{\leftarrow} + \mathbf{b}_h^{\leftarrow})$$
- ▶ We then define $\mathbf{h}_t = [\mathbf{h}_t^{\rightarrow}, \mathbf{h}_t^{\leftarrow}]$ as the state representation at time t , capturing both past and future information.

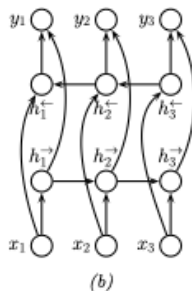
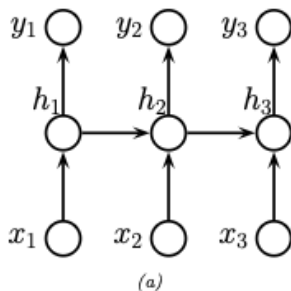
Bidirectional RNN: classifier

- We average pool over the hidden states across time to get the final classifier: $p(y|x_{1:T}) = \text{Cat}(y | \text{softmax}(\mathbf{W}\bar{\mathbf{h}}))$,
 $\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^T \mathbf{h}_t$



Seq2Seq: Sequence Translation

- ▶ Seq2seq models learn functions of the form $f_{\theta} : \mathbb{R}^{TD} \rightarrow \mathbb{R}^{T'C}$, used for tasks like sequence translation.
- ▶ There are two cases: an **aligned** case, where $T' = T$ and the input and output sequences have the same length, and an **unaligned** case, where $T' \neq T$.



Seq2Seq: Aligned Case

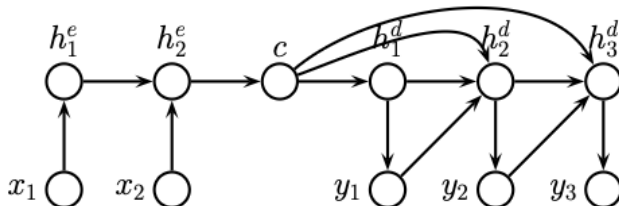
- ▶ In the aligned case, the input and output sequences line up one-to-one; we can think of this as **dense sequence labeling**, predicting one label per position, giving:

$$p(\mathbf{y}_{1:T}|\mathbf{x}_{1:T}) = \sum_{\mathbf{h}_{1:T}} \prod_{t=1}^T p(\mathbf{y}_t|\mathbf{h}_t) \mathbb{I}(\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t))$$

- ▶ The initial state is $\mathbf{h}_1 = f(\mathbf{h}_0, \mathbf{x}_1) = f_0(\mathbf{x}_1)$.
- ▶ $\mathbf{y}_t$ depends on $\mathbf{h}_t$, which only depends on the past inputs $\mathbf{x}_{1:t}$, with no access to the future.
- ▶ We can get better results by letting the decoder look into the future of $\mathbf{x}$ as well as the past, again using a bidirectional RNN.

Seq2Seq: Unaligned Case

- ▶ In the unaligned case, we map a sequence of length T to another of length T' .
- ▶ We first encode the input sequence into a context vector, $\mathbf{c} = f_e(\mathbf{x}_{1:T})$, using the last state of an RNN, or average pooling over a bidirectional RNN.
- ▶ We then generate the output sequence from this context vector using a decoder RNN, $\mathbf{y}_{1:T'} = f_d(\mathbf{c})$.
- ▶ Combining an encoder and decoder this way is called an **encoder-decoder architecture** (encode the input sequence to $\mathbf{c}$ and then decode from $\mathbf{c}$ to produce the output)
- ▶ Important application: **neural machine translation**

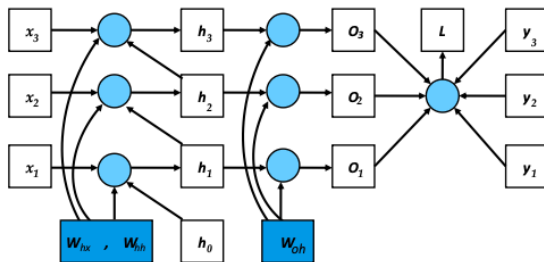


Teacher Forcing and Scheduled Sampling

- ▶ When training a language model, the likelihood of a sequence of words $w_1, w_2, \dots, w_T$ is $p(\mathbf{w}_{1:T}) = \prod_{t=1}^T p(w_t | \mathbf{w}_{1:t-1})$
- ▶ In an RNN, we set the input to $x_t = w_{t-1}$ and the output to $y_t = w_t$.
- ▶ We condition on the *ground truth* labels from the past, not on labels generated by the model; this is called **teacher forcing**.
- ▶ Teacher forcing can lead to poor test-time performance, because the model has only ever seen “correct” inputs during training, and may not know what to do if the input deviates from this at test time.
- ▶ A common fix is **scheduled sampling**: start with teacher forcing, but at random time steps feed in samples from the model instead, gradually increasing this fraction over training.

Backpropagation Through Time (BPTT)

- ▶ Fitting an RNN by MLE means solving $\theta^* = \arg \max_{\theta} p(\mathbf{y}_{1:T} | \mathbf{x}_{1:T}, \theta)$, which requires gradients of the loss with respect to the parameters.
- ▶ We compute these by unrolling the RNN's computation graph over time and applying standard backpropagation; this is called **backpropagation through time (BPTT)**.



An RNN unrolled (vertically) for 3 time steps, with the target output sequence and loss node. Copied from Figure 8.7.2 of [PML](#).

BPTT: Chain Rule

- ▶ Consider the simplified model $\mathbf{h}_t = \mathbf{W}_{hx}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1}$
 $\mathbf{o}_t = \mathbf{W}_{ho}\mathbf{h}_t$ with loss $L = \frac{1}{T} \sum_{t=1}^T \ell(y_t, \mathbf{o}_t)$.
- ▶ Using the shorthand $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)$ and $\mathbf{o}_t = g(\mathbf{h}_t, \mathbf{w}_o)$,
the chain rule gives

$$\frac{\partial L}{\partial \mathbf{w}_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial \ell(y_t, \mathbf{o}_t)}{\partial \mathbf{w}_h} = \frac{1}{T} \sum_{t=1}^T \frac{\partial \ell(y_t, \mathbf{o}_t)}{\partial \mathbf{o}_t} \frac{\partial g(\mathbf{h}_t, \mathbf{w}_o)}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h}$$

- ▶ The last term expands as

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{t-1}} \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{w}_h}$$

- ▶ This expression is recursive, since it depends on the same derivative evaluated at the previous time step.

BPTT: Recursive Expansion

- ▶ Expanding the recursion fully gives

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{w}_h} = \frac{\partial f(\mathbf{x}_t, \mathbf{h}_{t-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(\mathbf{x}_j, \mathbf{h}_{j-1}, \mathbf{w}_h)}{\partial \mathbf{h}_{j-1}} \right) \frac{\partial f(\mathbf{x}_i, \mathbf{h}_{i-1}, \mathbf{w}_h)}{\partial \mathbf{w}_h}$$

- ▶ This takes $O(T)$ time per time step, giving $O(T^2)$ overall for the full sequence.
- ▶ It is therefore standard to truncate the sum, keeping only the most recent K terms.

Truncated BPTT

- ▶ The truncation parameter K can be chosen adaptively, but is usually just set to the length of the subsequence in the current minibatch.
- ▶ Under truncated BPTT, we train on batches of short subsequences, typically non-overlapping windows extracted from the original sequence.
- ▶ If the previous subsequence ends at $t - 1$ and the current one starts at t , we can carry the hidden state over across batch updates during training.
- ▶ If the subsequences are not processed in order, we need to reset the hidden state instead.

Vanishing and Exploding Gradients

- ▶ Because we multiply by $\mathbf{W}_{hh}$ at each time step, activations in an RNN can decay or explode as we move forward in time.
- ▶ Similarly, gradients can decay or explode backwards in time, since we multiply Jacobians at each step during BPTT.
- ▶ A simple heuristic to address this is gradient clipping; more sophisticated methods try to control the spectral radius λ of $\mathbf{W}_{hh}$ and of the backward Jacobian $\mathbf{J}_{hh}$.
- ▶ The simplest way to control the spectral radius is to randomly initialize $\mathbf{W}_{hh}$ so that $\lambda \approx 1$, and keep it fixed rather than learning it.
- ▶ In that case, only the output matrix $\mathbf{W}_{ho}$ needs to be learned, making training a convex optimization problem; this is called an **echo state network**.

Gating and Long-Term Memory

- ▶ In principle, an RNN with enough hidden units could remember inputs from far in the past.
- ▶ In practice, “vanilla” RNNs fail to do this because of the vanishing gradient problem.
- ▶ The solution here is to update the hidden state additively, similar in spirit to the residual connections seen earlier in the course.
- ▶ The **gated recurrent unit (GRU)** lets the model learn *when to update* its hidden state, using a dedicated gating unit.
- ▶ This allows the model to selectively “remember” important information as soon as it’s seen.
- ▶ It also allows the model to learn when to reset the hidden state, forgetting information that’s no longer useful.

GRU: Reset and Update Gates

- ▶ Notation: $\mathbf{X}_t$ is an $N \times D$ matrix (N = batch size, D = vocabulary size), and $\mathbf{H}_t$ is an $N \times H$ matrix (H = number of hidden units).
- ▶ The **reset gate** $\mathbf{R}_t \in \mathbb{R}^{N \times H}$ and **update gate** $\mathbf{Z}_t \in \mathbb{R}^{N \times H}$ are computed as

$$\mathbf{R}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xr} + \mathbf{H}_{t-1} \mathbf{W}_{hr} + \mathbf{b}_r)$$

$$\mathbf{Z}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xz} + \mathbf{H}_{t-1} \mathbf{W}_{hz} + \mathbf{b}_z)$$

- ▶ Because of the sigmoid function, every entry of $\mathbf{R}_t$ and $\mathbf{Z}_t$ lies in $[0, 1]$, acting like a soft switch.

GRU: Candidate State

- ▶ The “candidate” next state vector is

$$\tilde{\mathbf{H}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xh} + (\mathbf{R}_t \odot \mathbf{H}_{t-1}) \mathbf{W}_{hh} + \mathbf{b}_h)$$

- ▶ This combines old memories that haven't been reset, $\mathbf{R}_t \odot \mathbf{H}_{t-1}$, with the new input $\mathbf{X}_t$, passed through a tanh to keep values in $(-1, 1)$.
- ▶ If the reset gate entries are close to 1, we recover the standard RNN update; if close to 0, the model behaves more like an MLP applied to $\mathbf{X}_t$.
- ▶ So the reset gate lets the model capture new, short-term information.

GRU: State Update

- ▶ The actual new state mixes the candidate state with the old state, using the update gate as a mixing weight:
$$\mathbf{H}_t = \mathbf{Z}_t \odot \mathbf{H}_{t-1} + (1 - \mathbf{Z}_t) \odot \tilde{\mathbf{H}}_t$$
- ▶ When $Z_{td} = 1$, the old value $H_{t-1,d}$ passes through unchanged, and $\mathbf{X}_t$ is ignored for that dimension.
- ▶ Thus the update gate lets the model capture long-term dependencies by choosing to preserve information over many steps.

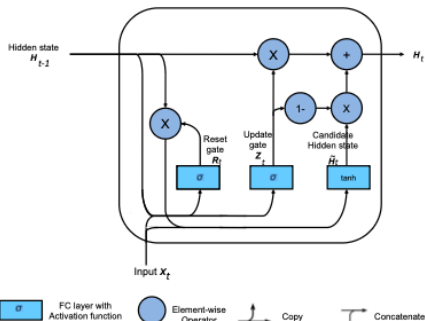


Illustration of a GRU. Copied from Figure 15.10 of [PML](#).

Long Short-Term Memory (LSTM)

- ▶ The **long short-term memory (LSTM)** model is a more sophisticated version of the GRU (see [Christopher Olah's blog post "Understanding LSTM Networks"](#)).
- ▶ The basic idea is to augment the hidden state $\mathbf{h}_t$ with a separate **memory cell** c_t .
- ▶ Three gates control the cell:
 - ▶ the **output gate** $\mathbf{O}_t$ (what gets read out):
$$\mathbf{O}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o)$$
 - ▶ the **input gate** $\mathbf{I}_t$ (what gets read in):
$$\mathbf{I}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i),$$
 - ▶ the **forget gate** $\mathbf{F}_t$ (when to reset the cell):
$$\mathbf{F}_t = \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f)$$

LSTM: Candidate and Actual Cell State

- ▶ We compute a candidate cell state using $\tanh$:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

- ▶ The actual cell update mixes this candidate with the old cell state, controlled by the input and forget gates:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t$$

- ▶ If $\mathbf{F}_t = 1$ and $\mathbf{I}_t = 0$, the cell carries its old value forward unchanged, which is what allows the LSTM to remember long-term information.

LSTM: Hidden State Output

- ▶ The hidden state is a transformed version of the cell, provided the output gate allows it through: $\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t)$
- ▶ $\mathbf{H}_t$ is used both as the output of the unit and as the hidden state for the next time step, giving the model a form of short-term memory.
- ▶ The cell $\mathbf{C}_t$ separately acts as the model's long-term memory.

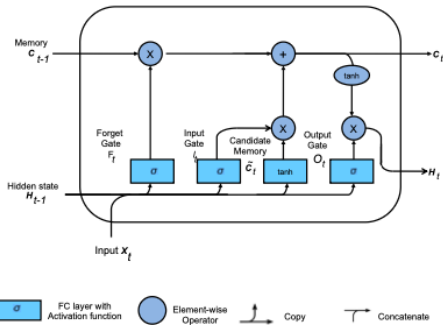


Illustration of a LSTM. Copied from Figure 15.11 of [PML](#).

Neural Networks for Sequences

- ▶ Introduction
- ▶ Recurrent Neural Networks (RNNs)
- ▶ **1d CNNs**
- ▶ Attention
- ▶ Transformers
- ▶ Language Models

1d CNNs

- ▶ Convolutional neural networks compute a function of some local neighborhood for each input using tied weights, and return an output.
- ▶ They are usually used for 2d inputs (images), but can also be applied in the 1d case, as we discuss here.
- ▶ They are an interesting alternative to RNNs, and are much easier to train, since they don't need to maintain a long-term hidden state.

1d CNNs for Sequence Classification

- ▶ We want to learn $f_{\theta} : \mathbb{R}^{DT} \rightarrow \mathbb{R}^C$, where T is the length of the input, D is the number of features per input, and C is the size of the output vector, e.g. class logits, i.e., we learn a mapping from variable-length sequences to a fixed-length output.
- ▶ Typically the input sequence has $D > 1$ input channels, or feature dimensions.
- ▶ We can convolve each channel separately and add up the result, using a different 1d filter (kernel) for each input channel:
$$z_i = \sum_d \mathbf{x}_{i-k:i+k,d}^T \mathbf{w}_d$$
 where k is the size of the 1d receptive field, and $\mathbf{w}_d$ is the filter for input channel d .
- ▶ This produces a 1d vector $\mathbf{z} \in \mathbb{R}^T$ encoding the input, ignoring boundary effects.
- ▶ We can create a vector representation at each location using a different weight vector for each output channel c :
$$z_{ic} = \sum_d \mathbf{x}_{i-k:i+k,d}^T \mathbf{w}_{d,c}$$
 which implements a mapping from TD to TC .

From Feature Maps to Class Logits

- ▶ To reduce the TC representation to a fixed-size vector $\mathbf{z} \in \mathbb{R}^C$, we can use max-pooling over time:

$$z_c = \max_i z_{ic}$$

- ▶ We then pass this pooled vector into a softmax layer to get class probabilities.
- ▶ Kim (2014) applied this model to sequence classification: each word is embedded, then various features are computed using 1d kernels of different widths, to capture patterns at different length scales.
- ▶ The results of max pooling over time across these different kernel widths are concatenated and passed to a fully connected layer.

Causal 1d CNNs for Sequence Generation

- ▶ To use 1d CNNs in a generative setting, we must convert them to a **causal CNN**, where each output variable only depends on previously generated variables.
- ▶ This is also called a **convolutional Markov model**.
- ▶ The model is defined as

$$p(\mathbf{y}) = \prod_{t=1}^T p(y_t | \mathbf{y}_{1:t-1}) = \prod_{t=1}^T \text{Cat}\left(y_t \mid \text{softmax}\left(\varphi\left(\sum_{\tau=1}^{t-k} \mathbf{w}^T \mathbf{y}_{\tau:\tau+k}\right)\right)\right)$$

where $\mathbf{w}$ is the convolutional filter of size k , assuming a single nonlinearity φ and categorical output for simplicity.

- ▶ This is called **causal convolution**: it is like a regular 1d convolution, except we “mask out” future inputs, so that y_t only depends on past values, not past and future values.

Neural Networks for Sequences

- ▶ Introduction
- ▶ Recurrent Neural Networks (RNNs)
- ▶ 1d CNNs
- ▶ **Attention**
- ▶ Transformers
- ▶ Language Models

Attention: Motivation

- ▶ In all the networks considered so far, hidden activations are a linear combination of input activations followed by a nonlinearity: $\mathbf{Z} = \varphi(\mathbf{XW})$, where $\mathbf{W}$ is a fixed set of weights learned on a training set.
- ▶ We can imagine a more flexible model in which the weights depend on the inputs: $\mathbf{Z} = \varphi(\mathbf{XW}(\mathbf{X}))$.
- ▶ This kind of **multiplicative interaction** is called **attention**.
- ▶ More generally, we write $\mathbf{Z} = \varphi(\mathbf{VW}(\mathbf{Q}, \mathbf{K}))$, where $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ are queries, keys, and values.

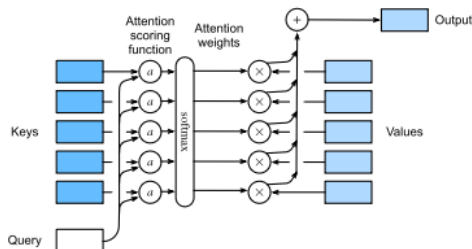
Attention: Queries, Keys, and Values

- ▶ $\mathbf{Q} \in \mathbb{R}^{m \times q}$ are **queries**, derived from $\mathbf{X}$, describing what each input is “looking for”.
- ▶ $\mathbf{K} \in \mathbb{R}^{m \times q}$ are **keys**, derived from $\mathbf{X}$, describing what each input vector contains.
- ▶ $\mathbf{V} \in \mathbb{R}^{m \times v}$ are **values**, derived from $\mathbf{X}$, describing how each input should be transmitted to the output.
- ▶ These are usually computed via linear projections: $\mathbf{Q} = \mathbf{W}_q \mathbf{X}$, $\mathbf{K} = \mathbf{W}_k \mathbf{X}$, $\mathbf{V} = \mathbf{W}_v \mathbf{X}$.

Attention: Computing Outputs

- ▶ To compute output $\mathbf{z}_j$, we compare its query $\mathbf{q}_j$ to each key $\mathbf{k}_i$ to get a similarity score $0 \leq \alpha_{ij} \leq 1$, with $\sum_i \alpha_{ij} = 1$, then set $\mathbf{z}_j = \sum_i \alpha_{ij} \mathbf{v}_i$.
- ▶ Example: if $\mathbf{V} = \mathbf{X}$ and query $\mathbf{q}_j$ matches keys 1 and 2 equally, so $\alpha_{1j} = \alpha_{2j} = 0.5$, then $\mathbf{z}_j = 0.5\mathbf{x}_1 + 0.5\mathbf{x}_2$.
- ▶ Outputs become a **dynamic** weighted combination of the inputs, rather than a fixed one; we learn the projection matrices $\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v$ instead of a single weight matrix.
- ▶ Attention was originally developed for natural language sequence models, but is nowadays applied to a variety of models, including vision models.

Attention: Illustration



Attention computes a weighted average of a set of values, where the weights are derived by comparing the query vector to a set of keys.

Copied from Figure 15.16 of [PML](#).

Attention as Soft Dictionary Lookup

- ▶ We can think of attention as a dictionary lookup, comparing a query $\mathbf{q}$ to each key $\mathbf{k}_i$ and retrieving the corresponding value $\mathbf{v}_i$.
- ▶ To make this differentiable, instead of retrieving a single value, we compute a convex combination of all the values:

$$\text{Attn}(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \dots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^m \alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) \mathbf{v}_i \in \mathbb{R}^v$$

- ▶ $\alpha_i(\mathbf{q}, \mathbf{k}_{1:m})$ is the i 'th **attention weight**, satisfying $0 \leq \alpha_i \leq 1$ and $\sum_i \alpha_i = 1$.
- ▶ These weights are computed from an **attention score** function $a(\mathbf{q}, \mathbf{k}_i) \in \mathbb{R}$, via softmax:

$$\alpha_i(\mathbf{q}, \mathbf{k}_{1:m}) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^m \exp(a(\mathbf{q}, \mathbf{k}_j))}$$

Masked Attention

- ▶ In some cases we want to restrict attention to a subset of the dictionary, corresponding to valid entries – for example, when padding sequences to a fixed length for efficient minibatching.
- ▶ We “mask out” the padded locations; this is called **masked attention**.
- ▶ We implement this efficiently by setting the attention score for masked entries to a large negative number, such as -10^6 , so the corresponding softmax weights become 0.
- ▶ This is analogous to the causal convolution discussed earlier for 1d CNNs.

Kernel Regression as Non-parametric Attention

- ▶ We can interpret attention in terms of **kernel regression**, a nonparametric model where the predicted output at query point $\mathbf{x}$ is a weighted combination of all target labels y_i :

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i(\mathbf{x}, \mathbf{x}_{1:n}) y_i$$

- ▶ Here $\alpha_i(\mathbf{x}, \mathbf{x}_{1:n}) \geq 0$ measures the normalized similarity of test input $\mathbf{x}$ to training input $\mathbf{x}_i$.
- ▶ This similarity is usually computed via a density kernel, such as the Gaussian:

$$\mathcal{K}_\sigma(u) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}u^2}$$

where σ is the bandwidth, and we define $a(\mathbf{x}, \mathbf{x}_i) = \mathcal{K}_\sigma(\mathbf{x} - \mathbf{x}_i)$.

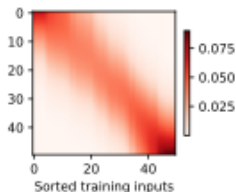
Kernel Regression: Connection to Attention

- ▶ Since the scores are normalized, we can drop the constant term and rewrite the kernel using $\beta^2 = 1/\sigma^2$: $\mathcal{K}(u; \beta) = \exp(-\frac{\beta^2}{2} u^2)$.
- ▶ Plugging this in gives

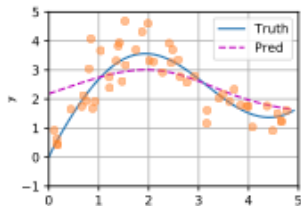
$$\begin{aligned} f(x) &= \sum_{i=1}^n \alpha_i(x, x_{1:n}) y_i \\ &= \sum_{i=1}^n \frac{\exp\left[-\frac{1}{2}((x - x_i)\beta)^2\right]}{\sum_{j=1}^n \exp\left[-\frac{1}{2}((x - x_j)\beta)^2\right]} y_i \\ &= \sum_{i=1}^n \text{softmax}_i \left[-\frac{1}{2}((x - x_1)\beta)^2, \dots, -\frac{1}{2}((x - x_n)\beta)^2 \right] y_i \end{aligned}$$

- ▶ This is a form of nonparametric attention, where the queries are test points $\mathbf{x}$, the keys are training inputs $\mathbf{x}_i$, and the values are training labels y_i .

Example Kernel Regression and Attention



(a)



(b)

Kernel regression in 1d. (a) Kernel weight (attention) matrix $A_{ji} = \alpha_i(\mathbf{x}_j, \mathbf{x}_{1:n})$, $\beta = 1$. The sparsity of the attention mechanism depends on β : increasing β (reducing the bandwidth) narrows the attention band, but the model starts to overfit. (b) Resulting predictions on a dense grid of test points, $\beta = 1$. Copied from Figure 15.17 of [PML](#).

Parametric Attention

- ▶ Non-parametric methods do not scale well to large training sets or high-dimensional inputs, so we turn to parametric models with a fixed set of keys and values, comparing queries and keys in a learned embedding space.
- ▶ In the general case, query $\mathbf{q} \in \mathbb{R}^q$ and key $\mathbf{k} \in \mathbb{R}^k$ may have different sizes; we map both to a common embedding space via $\mathbf{W}_q \mathbf{q}$ and $\mathbf{W}_k \mathbf{k}$.
- ▶ Passing these into an MLP gives the **additive attention** scoring function:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^T \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k}) \in \mathbb{R}$$

Scaled Dot-Product Attention

- ▶ A more efficient approach assumes queries and keys both have length d , so we can compute $\mathbf{q}^\top \mathbf{k}$ directly.
- ▶ If $\mathbf{q}$ and $\mathbf{k}$ are independent, zero-mean, unit-variance random variables, their inner product has mean 0 and variance d .
- ▶ To keep the variance at 1 regardless of input size, it's standard to divide by $\sqrt{d}$, giving **scaled dot-product attention**:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d} \in \mathbb{R}$$

- ▶ For minibatches of n query vectors, with matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$, we compute the attention-weighted outputs as

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} \right) \mathbf{V} \in \mathbb{R}^{n \times v}$$

where softmax is applied row-wise.

Seq2Seq with Attention: Motivation

- ▶ Recall the seq2seq model, using an RNN decoder
 $\mathbf{h}_t^d = f_d(\mathbf{h}_{t-1}^d, \mathbf{y}_{t-1}, \mathbf{c})$, with $\mathbf{c}$ a fixed-length context vector encoding the input $\mathbf{x}_{1:T}$.
- ▶ Usually $\mathbf{c} = \mathbf{h}_T^e$, the final state of the encoder RNN, or an average-pooled bidirectional RNN.
- ▶ For tasks like machine translation, this can give poor performance, since the output has no direct access to the input words themselves.
- ▶ We can avoid this bottleneck by letting the output words directly “look at” the input words – but since word order isn’t always preserved across languages, we need to infer the **alignment** between source and target.
- ▶ This alignment problem can be solved, in a differentiable way, using soft **attention**.

Seq2Seq with Attention: Model

- ▶ We replace the fixed context vector $\mathbf{c}$ in the decoder with a dynamic context vector $\mathbf{c}_t$:

$$\mathbf{c}_t = \sum_{i=1}^T \alpha_i(\mathbf{h}_{t-1}^d, \mathbf{h}_{1:T}^e) \mathbf{h}_i^e$$

- ▶ Here the query is the decoder's previous hidden state $\mathbf{h}_{t-1}^d$, and the keys and values are both the encoder's hidden states.
- ▶ When the RNN has multiple layers, we typically use the top layer of the encoder as keys/values, and the top layer of the decoder as the query.
- ▶ The context vector is concatenated with the decoder input $\mathbf{y}_{t-1}$ and fed into the decoder, with the previous hidden state, to create $\mathbf{h}_t^d$.
- ▶ Attention weights can be visualized at each decoding step, to see which parts of the input the model considers relevant for each output.

Soft vs Hard Attention

- ▶ If we force the attention heatmap to be sparse, so each output attends to only one input location rather than a weighted combination of all of them, the method is called **hard attention**.
- ▶ Hard attention results in a nondifferentiable training objective, requiring methods such as reinforcement learning to fit the model.
- ▶ Attention heatmaps seem able to “explain” why a model generates a given output, but the interpretability of attention remains a controversial topic.

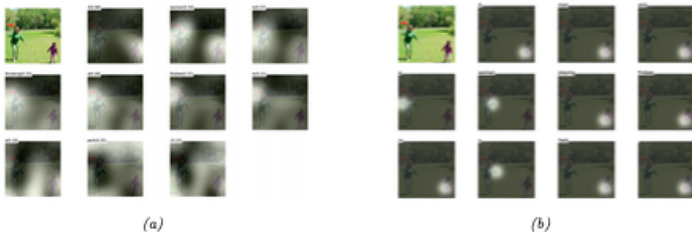


Image captioning using attention. (a) Soft attention. Generates “a woman is throwing a frisbee in a park”. (b) Hard attention. Generates “a man and a woman playing frisbee in a field”. Copied from Figure 15.22 of [PML](#).

Neural Networks for Sequences

- ▶ Introduction
- ▶ Recurrent Neural Networks (RNNs)
- ▶ 1d CNNs
- ▶ Attention
- ▶ **Transformers**
- ▶ Language Models

Transformers

- ▶ The **transformer** is a fairly complex seq2seq model that uses attention in both the encoder and the decoder, eliminating the need for RNNs entirely.
- ▶ Transformers have been used for many conditional sequence generation tasks: machine translation, constituency parsing, music generation, protein sequence generation, abstractive text summarization, and even image generation.

Self-Attention

- ▶ Instead of the decoder attending to the encoder, we can modify the model so the encoder attends to **itself** – this is called **self-attention**.
- ▶ Given a sequence of input tokens $\mathbf{x}_1, \dots, \mathbf{x}_n$, with $\mathbf{x}_i \in \mathbb{R}^d$, self-attention generates a sequence of outputs of the same size:

$$\mathbf{y}_i = \text{Attn}(\mathbf{x}_i, (\mathbf{x}_1, \mathbf{x}_1), \dots, (\mathbf{x}_n, \mathbf{x}_n))$$

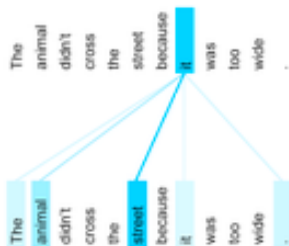
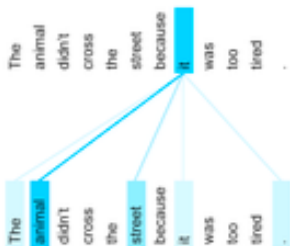
where the query is $\mathbf{x}_i$, and the keys and values are all the valid inputs $\mathbf{x}_1, \dots, \mathbf{x}_n$.

- ▶ To use self-attention in a decoder, we set $\mathbf{x}_i = \mathbf{y}_{i-1}$ and $n = i - 1$, so all previously generated outputs are available.
- ▶ At training time, all outputs are already known, so we can evaluate this function in parallel – overcoming the sequential bottleneck of RNNs.
- ▶ Self-attention can also give improved representations of context, useful for **coreference resolution**.

Self-Attention: Example

Translating “The animal didn’t cross the street because it was too *tired*” vs. “... too *wide*” into French requires knowing what “it” refers to in each case.

- In the first sentence, the representation for “it” depends on the earlier representation of “animal”; in the second, it depends on “street”.



Multi-Headed Attention

- ▶ If we think of an attention matrix as similar to a kernel matrix, it's natural to want multiple attention matrices, each capturing a different notion of similarity – this is **multi-headed attention (MHA)**.
- ▶ Given a query $\mathbf{q} \in \mathbb{R}^{d_q}$, keys $\mathbf{k}_j \in \mathbb{R}^{d_k}$, and values $\mathbf{v}_j \in \mathbb{R}^{d_v}$, the i 'th attention head is defined as

$$\mathbf{h}_i = \text{Attn}(\mathbf{W}_i^{(q)} \mathbf{q}, \{\mathbf{W}_i^{(k)} \mathbf{k}_j, \mathbf{W}_i^{(v)} \mathbf{v}_j\}) \in \mathbb{R}^{p_v}$$

where $\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)}$ are projection matrices.

- ▶ We stack the h heads together and project to $\mathbb{R}^{p_o}$:

$$\mathbf{h} = \text{MHA}(\mathbf{q}, \{\mathbf{k}_j, \mathbf{v}_j\}) = \mathbf{W}_o \begin{pmatrix} \mathbf{h}_1 \\ \vdots \\ \mathbf{h}_h \end{pmatrix} \in \mathbb{R}^{p_o}$$

- ▶ If we set $p_q h = p_k h = p_v h = p_o$, we can compute all the output heads in parallel.

Positional Encoding: Motivation

- ▶ The performance of “vanilla” self-attention can be low, since attention is permutation invariant and hence ignores word order.
- ▶ To overcome this, we concatenate the word embeddings with a **positional embedding**, so the model knows what order the words occur in.
- ▶ One approach is to represent each position by an integer; but since neural networks can't natively handle integers, we can encode the integer in binary form.
- ▶ Example: for sequence length $n = 8$, use $d = 3$ -dimensional bit vectors per location: 000, 001, 010, $\dots$, 111; the rightmost index toggles fastest, the leftmost slowest.
- ▶ We can represent this as a position matrix $\mathbf{P} \in \mathbb{R}^{n \times d}$.

Positional Encoding: Sinusoidal Basis

- ▶ We can think of the binary representation as using basis functions (powers of 2) with 0/1 coefficients; a more compact code uses different basis functions with real-valued weights.
- ▶ Vaswani et al. propose a sinusoidal basis:

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/d}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/d}}\right)$$

where $C = 10,000$ corresponds to some maximum sequence length.

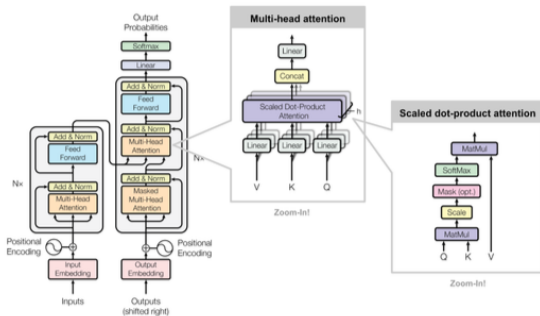
Positional Encoding: Key Properties

- ▶ This representation has two advantages: it can be computed for arbitrary-length inputs (up to $T \leq C$), unlike a learned mapping from integers to vectors.
- ▶ The representation of one location is also linearly predictable from any other, given their relative distance: $\mathbf{p}_{t+\phi} = f(\mathbf{p}_t)$, for a linear transformation f .
- ▶ Since this amounts to a rotation, if ϕ is small then $\mathbf{p}_{t+\phi} \approx \mathbf{p}_t$.
- ▶ Once computed, the positional embeddings $\mathbf{P}$ are combined with the word embeddings $\mathbf{X}$ via

$$\text{POS}(\text{Embed}(\mathbf{X})) = \mathbf{X} + \mathbf{P}$$

Putting It All Together: Encoder Block

- ▶ A transformer is a seq2seq model that uses self-attention for the encoder and decoder rather than an RNN.
- ▶ The **encoder** uses a series of encoder blocks, each combining multi-headed attention, residual connections, feedforward layers, and layer normalization (LHS, plot below).
- ▶ The **decoder** attends to the encoder via a second multi-head attention block, and also attends to previously generated outputs via a masked (causal) multi-head attention block (RHS, plot below)



- ▶ Introduction
- ▶ Recurrent Neural Networks (RNNs)
- ▶ 1d CNNs
- ▶ Attention
- ▶ Transformers
- ▶ **Language Models**

Language Models and Unsupervised Representation Learning

- ▶ RNNs and autoregressive (decoder-only) transformers can be used as **language models**: generative sequence models of the form $p(x_1, \dots, x_T) = \prod_{t=1}^T p(x_t | \mathbf{x}_{1:t-1})$, where each x_t is a discrete token, such as a word or wordpiece.
- ▶ The latent state of these models can be used as a continuous vector representation of the text – instead of a one-hot vector $\mathbf{x}_t$, we use the hidden state $\mathbf{h}_t$, which depends on all the previous words in the sentence.
- ▶ These vectors are called **contextual word embeddings**, useful for text classification or seq2seq tasks.
- ▶ The advantage of this approach is that we can **pre-train** the language model in an unsupervised way on a large corpus of text, and then **fine-tune** it in a supervised way on a small labeled task-specific dataset; this general approach is called **transfer learning**.

Generative vs Non-Causal Models

- ▶ If our primary goal is to compute useful representations for transfer learning, rather than generating text, we can replace the generative sequence model with non-causal models that compute a representation of a sentence but cannot generate it.
- ▶ These models have the advantage that the hidden state $\mathbf{h}_t$ can depend on the past $\mathbf{y}_{1:t-1}$, present $\mathbf{y}_t$, and future $\mathbf{y}_{t+1:T}$, which can sometimes result in better representations, since it takes into account more context.
- ▶ However, if our goal is to generate textual outputs, we must use a “causal” or generative language model.
- ▶ We discuss both variants below.

Non-generative Language Models: ELMo

- ▶ **ELMo** (“Embeddings from Language Model”) fits two RNN language models, one left-to-right and one right-to-left, and then combines their hidden state representations to produce an embedding for each word.
- ▶ Unlike a biRNN, which needs an input-output pair, ELMo is trained in an unsupervised way, minimizing the negative log likelihood of the input sentence $\mathbf{x}_{1:T}$:
$$\mathcal{L}(\theta) = - \sum_{t=1}^T [\log p(x_t | \mathbf{x}_{1:t-1}; \theta_e, \theta^{\rightarrow}, \theta_s) + \log p(x_t | \mathbf{x}_{t+1:T}; \theta_e, \theta^{\leftarrow}, \theta_s)]$$
 where θ_e are the shared embedding layer parameters, θ_s are the shared softmax output parameters, and $\theta^{\rightarrow}, \theta^{\leftarrow}$ are the parameters of the two LSTM RNNs.

ELMo: Illustration

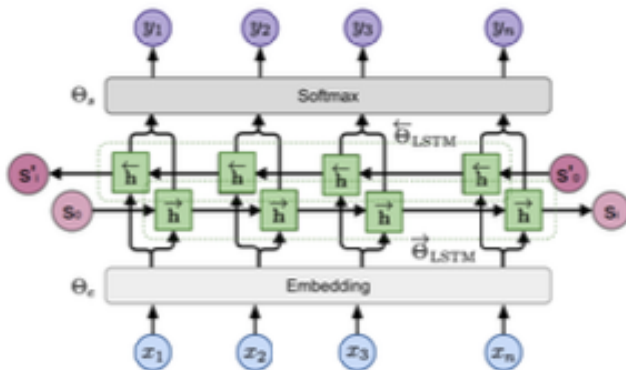


Illustration of ELMo bidirectional language model. Here $y_t = x_{t+1}$ when acting as the target for the forwards LSTM, and $y_t = x_{t-1}$ for the backwards LSTM. Copied from Figure 15.32 of [PML](#).

BERT

- ▶ **BERT** (Bidirectional Encoder Representations from Transformers) is, like ELMo, a non-causal model that creates representations of text but cannot generate text.
- ▶ It uses a transformer model to map a modified version of a sequence back to the unmodified form.
- ▶ The modified input at location t omits the t 'th token, and the task is to predict the missing word given the remaining words.
- ▶ This is called the **fill-in-the-blank**, or **cloze**, task.
- ▶ It is trained to minimize the negative log **pseudo-likelihood**:
 $\mathcal{L} = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \mathbb{E}_{\mathbf{m}} \sum_{i \in \mathbf{m}} -\log p(x_i | \mathbf{x}_{-\mathbf{m}})$ where $\mathbf{m}$ is a random binary mask.

BERT: Examples and Applications

- ▶ Example, training on cooking video transcripts: “Let’s make [MASK] chicken! [SEP] It [MASK] great with orange sauce.”, with target labels “some” and “tastes”.
- ▶ [SEP] is a separator token inserted between two sentences.
- ▶ The conditional probability is given by applying a softmax to the final layer hidden vector at location i :
$$p(x_i|\hat{\mathbf{x}}) = \frac{\exp(\mathbf{h}(\hat{\mathbf{x}})_i^T \mathbf{e}(x_i))}{\sum_{x'} \exp(\mathbf{h}(\hat{\mathbf{x}})_i^T \mathbf{e}(x'))}$$
 where $\hat{\mathbf{x}} = \mathbf{x}_{-m}$ is the masked input, similar to a denoising autoencoder.
- ▶ The model can be also used to classify if one sentence follows another (next sentence prediction task)
- ▶ It requires sophisticated tuning for better performance.

Generative (Causal) Large Language Models

- ▶ Generative, or causal, language models based on decoder-only transformers are generally known as **large language models (LLMs)**.
- ▶ **GPT** (“Generative Pre-training Transformer”) is a causal, generative model that uses a masked transformer as the decoder.
- ▶ The original GPT paper jointly optimizes on a large unlabeled dataset and a small labeled dataset: $\mathcal{L} = \mathcal{L}_{\text{cls}} + \lambda \mathcal{L}_{\text{LM}}$, where $\mathcal{L}_{\text{cls}}$ is the classification loss on labeled data and $\mathcal{L}_{\text{LM}}$ is the language modeling loss on unlabeled data.
- ▶ Following versions (**GPT-2**, **GPT-3**, **GPT-2x**) are larger version of GPT, trained on a large web corpus.

GPT: ChatGPT and RLHF

- ▶ **ChatGPT** is an improved version of GPT-x, trained to have interactive dialogs using a technique called **reinforcement learning from human feedback (RLHF)**.
- ▶ This technique was first introduced in the **InstructGPT** paper, which uses reinforcement learning to fine-tune the model so it generates responses that are more “aligned” with human intent.
- ▶ Alignment is estimated by a ranking model, which is itself pre-trained on supervised data.

Applications of LLMs

- ▶ LLMs like GPT can generate text given an initial input **prompt**; if the generated response fulfills a task “out of the box”, the model is performing **zero-shot task transfer**.
- ▶ Example – **abstractive summarization** (as opposed to **extractive summarization**, which just selects a subset of input words): sample from $p(\mathbf{x}_{T+1:T+100} | [\mathbf{x}_{1:T}; \mathbf{TL;DR}])$, where **TL;DR** is a special token appended to the input, signalling that a summary is wanted.
- ▶ TL;DR stands for “too long; didn’t read” and frequently occurs in web text followed by a human-created summary; adding this token “triggers” the transformer decoder into summarization mode – an example of **prompt engineering**.
- ▶ An arguably better way to tell the model what task to perform is to train it on input-output pairs directly.
- ▶ LLMs are used to build **chatbots**, such as ChatGPT, Claude, DeepSeek, Grok, for code and image generation and many other tasks.

References

- ▶ [PML](#): Chapter 15